

Nama : Rika bagus Wijaya
Nim : G.231.24.0001
Matkul : Analisis kompleksitas algoritma

Tugas praktikum 5

1. Silakan Sdr cari 4 metode [sorting](#).
2. Buatlah pseudo code dan coding python
3. Berikan analisa kompleksitasnya
4. Simpan dalam file pdf dengan nama Sorting_NIM.pdf

1. Bubble Sort

Bubble Sort merupakan salah satu metode pengurutan data yang paling sederhana karena cara kerjanya hanya membandingkan dua elemen yang letaknya bersebelahan lalu menukarnya apabila urutannya masih salah, proses tersebut dilakukan berulang kali dari awal hingga akhir array sampai tidak ada lagi data yang perlu ditukar sehingga seluruh data menjadi terurut. Algoritma ini dinamakan “bubble” karena nilai terbesar akan bergerak sedikit demi sedikit ke bagian akhir seperti gelembung yang naik ke permukaan air. Walaupun mudah dipahami dan cocok digunakan untuk pembelajaran dasar algoritma, Bubble Sort kurang efisien ketika digunakan pada jumlah data yang besar karena membutuhkan banyak proses perbandingan dan pertukaran data.

Pseudo Code

```
BubbleSort(A)
  n ← panjang array

  for i ← 0 to n-1
    for j ← 0 to n-i-2
      if A[j] > A[j+1]
        tukar A[j] dengan A[j+1]
```

Coding Python

```
def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n - i - 1):

            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

```
return arr
```

```
data = [64, 34, 25, 12, 22, 11, 90]
```

```
print("Hasil Bubble Sort :", bubble_sort(data))
```

Dari sisi kompleksitas waktu, Bubble Sort memiliki best case sebesar $O(n)$ apabila data sebenarnya sudah hampir terurut sehingga proses pertukaran sangat sedikit, sedangkan average case dan worst case berada pada $O(n^2)$ karena algoritma tetap melakukan banyak perbandingan antar elemen. Kompleksitas memorinya termasuk kecil yaitu $O(1)$ karena tidak membutuhkan ruang penyimpanan tambahan, sehingga algoritma ini sering dipilih untuk latihan dasar memahami konsep sorting meskipun performanya tidak terlalu baik untuk data berukuran besar.

2. Selection Sort

Selection Sort adalah metode pengurutan yang bekerja dengan mencari nilai terkecil dari seluruh data yang belum terurut kemudian menempatkannya ke posisi yang benar secara bertahap. Pada setiap iterasi, algoritma akan memilih satu elemen paling kecil lalu menukarnya dengan elemen pada posisi awal bagian array yang belum tersusun. Proses ini terus dilakukan sampai semua data tersusun rapi dari nilai terkecil ke terbesar. Cara kerja algoritma ini cukup mudah dipahami karena mirip seperti seseorang yang mencari buku paling tipis dari tumpukan lalu meletakkannya di bagian paling depan satu per satu hingga seluruh buku tersusun.

Pseudo Code

SelectionSort(A)

$n \leftarrow$ panjang array

 for $i \leftarrow 0$ to $n-1$

$\text{min} \leftarrow i$

 for $j \leftarrow i+1$ to $n-1$

 if $A[j] < A[\text{min}]$

$\text{min} \leftarrow j$

 tukar $A[i]$ dengan $A[\text{min}]$

Coding Python

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        min_index = i
```

```

for j in range(i + 1, n):

    if arr[j] < arr[min_index]:
        min_index = j

arr[i], arr[min_index] = arr[min_index], arr[i]

return arr

```

```
data = [64, 25, 12, 22, 11]
```

```
print("Hasil Selection Sort :", selection_sort(data))
```

Selection Sort memiliki kompleksitas waktu $O(n^2)$ pada kondisi terbaik, rata-rata, maupun terburuk karena algoritma tetap harus mencari elemen terkecil pada setiap proses iterasi tanpa memperhatikan apakah data sudah hampir terurut atau belum. Walaupun jumlah pertukarannya lebih sedikit dibanding Bubble Sort, proses pencarian minimum yang dilakukan berulang membuat algoritma ini tetap kurang efisien untuk data dalam jumlah besar. Namun, penggunaan memori pada algoritma ini cukup hemat karena hanya membutuhkan kompleksitas ruang $O(1)$.

3. Insertion Sort

Insertion Sort adalah metode pengurutan yang bekerja dengan cara mengambil satu elemen lalu menyisipkannya ke posisi yang tepat pada bagian data yang sudah terurut. Cara kerjanya sangat mirip seperti seseorang menyusun kartu remi di tangan, di mana setiap kartu baru akan ditempatkan pada posisi yang sesuai agar urutan kartu tetap rapi. Algoritma ini dimulai dari elemen kedua karena elemen pertama dianggap sudah terurut, kemudian setiap elemen berikutnya dibandingkan dengan data sebelumnya dan digeser apabila posisinya belum sesuai.

Pseudo Code

InsertionSort(A)

$n \leftarrow$ panjang array

 for $i \leftarrow 1$ to $n-1$

 key $\leftarrow A[i]$

$j \leftarrow i-1$

 while $j \geq 0$ dan $A[j] > \text{key}$

$A[j+1] \leftarrow A[j]$

$j \leftarrow j-1$

$A[j+1] \leftarrow \text{key}$

Coding Python

```
def insertion_sort(arr):
```

```
    for i in range(1, len(arr)):
```

```
        key = arr[i]
```

```
        j = i - 1
```

```
        while j >= 0 and arr[j] > key:
```

```
            arr[j + 1] = arr[j]
```

```
            j -= 1
```

```
        arr[j + 1] = key
```

```
    return arr
```

```
data = [12, 11, 13, 5, 6]
```

```
print("Hasil Insertion Sort :", insertion_sort(data))
```

Dari sisi performa, Insertion Sort memiliki best case $O(n)$ ketika data sudah hampir tersusun sehingga hanya memerlukan sedikit pergeseran data. Namun pada kondisi rata-rata dan terburuk, kompleksitas waktunya menjadi $O(n^2)$ karena banyak elemen yang harus dipindahkan ke posisi yang benar. Walaupun demikian, algoritma ini cukup efisien untuk data berukuran kecil dan sering digunakan karena implementasinya sederhana serta tidak membutuhkan memori tambahan yang besar.

4. Merge Sort

Merge Sort merupakan algoritma pengurutan yang menggunakan pendekatan divide and conquer, yaitu membagi data menjadi bagian-bagian kecil terlebih dahulu kemudian menggabungkannya kembali dalam keadaan terurut. Proses pembagian dilakukan terus menerus sampai array hanya berisi satu elemen karena data dengan satu elemen dianggap sudah pasti terurut. Setelah itu, algoritma mulai menggabungkan kembali bagian-bagian kecil tersebut sambil membandingkan nilainya agar menghasilkan urutan yang benar. Pendekatan ini membuat Merge Sort jauh lebih efisien dibanding algoritma sorting sederhana lainnya ketika digunakan pada data dalam jumlah besar.

Pseudo Code

MergeSort(A)

 jika panjang A > 1

 bagi array menjadi kiri dan kanan

 MergeSort(kiri)

 MergeSort(kanan)

 gabungkan kiri dan kanan secara terurut

Coding Python

```
def merge_sort(arr):
```

```
    if len(arr) > 1:
```

```
        mid = len(arr) // 2
```

```
        left = arr[:mid]
```

```
        right = arr[mid:]
```

```
        merge_sort(left)
```

```
        merge_sort(right)
```

```
    i = j = k = 0
```

```
    while i < len(left) and j < len(right):
```

```
        if left[i] < right[j]:
```

```
            arr[k] = left[i]
```

```
            i += 1
```

```
        else:
```

```
            arr[k] = right[j]
```

```
            j += 1
```

```
    k += 1
```

```
    while i < len(left):
```

```
        arr[k] = left[i]
```

```
        i += 1
```

```
        k += 1
```

```
    while j < len(right):
```

```
arr[k] = right[j]  
j += 1  
k += 1
```

```
return arr
```

```
data = [38, 27, 43, 3, 9, 82, 10]
```

```
print("Hasil Merge Sort :", merge_sort(data))
```

Merge Sort memiliki kompleksitas waktu $O(n \log n)$ pada kondisi terbaik, rata-rata, maupun terburuk sehingga performanya jauh lebih stabil dibanding Bubble Sort, Selection Sort, dan Insertion Sort. Hal tersebut terjadi karena proses pembagian data dan penggabungan dilakukan secara sistematis menggunakan rekursi. Kekurangan utama algoritma ini adalah kebutuhan memori tambahan sebesar $O(n)$ karena proses penggabungan membutuhkan array sementara, namun secara keseluruhan Merge Sort termasuk salah satu algoritma sorting yang sangat efisien dan sering digunakan pada sistem pengolahan data berukuran besar.